

TECHNICAL AUDIT REPORT

VIXOR App

Comprehensive Code Review

Full verification of 3 previously modified files, combined source reference, and complete issue taxonomy for KIMI K3 remediation.

PROJECT	STACK	DATE	BRANCH
VIXOR Trading Dashboard	React / TanStack Start	2026-07-19	main

Table of Contents

1. Executive Summary	4
2. Fix Verification - Modified Files Analysis	5
2.1 AssetRegistry (types.ts) - VERIFIED FIXED	5
2.2 useLivePrices Hook - VERIFIED FIXED	5
2.3 Analysis Runner - VERIFIED FIXED	5
2.4 Fourth File: openrouter_free_models.json	6
3. Complete Issue Taxonomy	7
3.1 Critical Issues (Fix Immediately)	7
3.2 High Severity Issues	7
3.3 Medium Severity Issues	8
3.4 Low and Informational Issues	8
4. Original 13 Issue Categories - Status Mapping	9
5. Security Vulnerabilities	10
5.1 Authentication Bypass	10
5.2 Cross-User Data Leak	10
5.3 Missing Auth Middleware on Endpoints	10
5.4 Ineffective Rate Limiting	10
6. Architecture and Design Issues	10
6.1 Swap Component - Complete Simulation	11
6.2 Mock Data Proliferation	11
6.3 Analysis Pipeline Fragility	11
7. Actionable Prompts for KIMI K3	12
7.1 CRITICAL: Fix Authentication Bypass	12
7.2 CRITICAL: Fix Watchlist Data Leak	12
7.3 CRITICAL: Fix or Disable Swap Component	12
7.4 HIGH: Fix EquityChart Gradient ID Collision	13
7.5 HIGH: Fix scanArbitrage Stub	13
7.6 MEDIUM: Remove "Mock Data" Label from Discover	13
7.7 MEDIUM: Add Auth Middleware to Server Functions	13

7.8 MEDIUM: Add Analysis Polling Timeout	13
8. Architecture Recommendations for Clean Structure	14
8.1 Remove All Mock Data from Production Paths	14
8.2 Centralize Authentication Middleware	14
8.3 Implement Proper Error Boundaries and Fallbacks	14
8.4 Data Layer Consolidation	14
9. Build and CI Notes for KIMI K3	14

1. Executive Summary

This report provides a comprehensive technical audit of the VIXOR crypto trading dashboard application, a React/TanStack Start based Telegram Web App. The audit was triggered by user-reported issues across 13 categories identified from application screenshots, covering UI inconsistencies, mock data usage, missing real-time data, broken analysis pipelines, and security vulnerabilities. Three files were previously modified in attempted fixes, and this report verifies the current state of those changes while cataloging all remaining and newly discovered issues.

The VIXOR application is built on a modern stack including React, TanStack Start, TanStack Query, Binance WebSocket for real-time crypto prices, TwelveData for forex/commodity data, OpenRouter AI for chart analysis, and Supabase for persistence. The design system uses a dark theme with primary color #6366F1 (indigo), bullish #22D3A6 (green), bearish #FB4667 (red), and gold #F0C419 accents. The app targets Telegram Web App deployment, which imposes specific constraints on viewport size and navigation patterns.

The audit found that while 3 backend fixes were successfully applied and persisted in the codebase, they address only a small fraction of the overall problem surface. The three verified fixes are: (1) LINK/USDT added to AssetRegistry, (2) useLivePrices changed from .get() to .find() for resilience, and (3) OpenRouter model ID corrected with local fallback engine. However, 5 CRITICAL issues, 3 HIGH issues, 7 MEDIUM issues, and numerous LOW/INFO issues remain unaddressed across the application, spanning mock data in production pages, security vulnerabilities in authentication middleware, data leak between users, and broken UI components.

Metric	Value
Files Verified as Fixed	3 of 3
Total Issues Found	20+
Critical Issues	5
High Severity	3
Medium Severity	7+
13 Original Categories Resolved	0 of 13

Table 1: Audit Summary Statistics

2. Fix Verification - Modified Files Analysis

Three files were modified across three fix commits: 8c859bd (AssetRegistry + useLivePrices), b6076e6 (OpenRouter model ID), and a6e1fe2 (local fallback engine). A fourth file, download/openrouter_free_models.json, was a data lookup file, not a source code fix. This section provides a detailed verification of each fix, confirming what was actually applied versus what the user suspected might have been lost.

2.1 AssetRegistry (types.ts) - VERIFIED FIXED

File: src/shared/asset-registry/types.ts

Commit: 8c859bd

Status: **FIXED AND PERSISTED**

The LINK/USDT asset entry has been successfully added to the ASSETS array at lines 347-364. The entry includes complete symbol mappings for Binance (LINKUSDT), TwelveData (LINK/USDT), and TradingView (BINANCE:LINKUSDT), along with proper configuration parameters including pipSize of 0.01, 2 decimal places, volatility of 0.03, typicalRange of 0.04, and a basePrice of 18. The asset is correctly marked as active and popular with priority 8, ensuring it appears in quick-select lists and signal generation.

Additionally, a new .find(pair) method was added to the AssetRegistryClass at line 693, which returns AssetDefinition | undefined instead of throwing an error like the existing .get(pair) method. This provides a safe lookup path for callers that need to handle unknown pairs gracefully without try/catch blocks. The .get() method at line 686 still throws Error("[AssetRegistry] Unknown pair: ...") for strict validation scenarios.

2.2 useLivePrices Hook - VERIFIED FIXED

File: src/shared/market-data/use-live-prices.ts

Commit: 8c859bd

Status: **FIXED AND PERSISTED**

The critical crash bug has been fixed. At line 77, the code now uses AssetRegistry.find(pair) instead of the previous AssetRegistry.get(pair). The .find() method returns undefined for unknown pairs instead of throwing an exception. At line 78, the code gracefully skips unknown pairs with if (!asset) continue;, preventing the entire price feed from crashing when an unregistered pair (like LINK/USDT before the fix) is requested.

The same pattern is applied consistently at line 156 and line 193, where AssetRegistry.find() is used with proper undefined checks. The fix is architecturally correct: it makes the hook resilient to registry gaps without silently hiding errors. Unknown pairs are simply excluded from the live price feed rather than crashing the entire application.

2.3 Analysis Runner - VERIFIED FIXED

File: src/domains/analysis/server/run-analysis.ts

Commits: b6076e6 + a6e1fe2

Status: **BOTH FIXES PERSISTED**

Two fixes were applied to this file and both are confirmed present. First, at line 226, the OpenRouter model ID was changed from the invalid google/gemini-2.0-flash-lite-preview-02-05:free to

google/gemma-4-31b-it:free, read from process.env.OPENROUTER_MODEL with the new model as the default fallback. This ensures the analysis pipeline uses a valid, accessible free model from OpenRouter.

Second, a comprehensive fallback mechanism was added. At lines 180-190, when no OPENROUTER_API_KEY environment variable is set, the system immediately falls back to the local analysis engine via runLocalAnalysisFallback() instead of crashing. Additionally, at lines 307-319, a try/catch block wraps the entire OpenRouter API call, so if the API request fails (network error, rate limit, invalid response), the system gracefully degrades to the local engine. The local fallback at lines 326-408 includes its own error handling: if even the local engine fails, it generates a synthetic fallback result using generateFallbackResult(). This creates a three-tier resilience: OpenRouter API, local analysis engine, synthetic fallback.

2.4 Fourth File: openrouter_free_models.json

File: download/openrouter_free_models.json

Commit: b6076e6

Status: Data file (not source code)

This file is a JSON data dump of available free models from the OpenRouter API, used as a reference during the model ID fix investigation. It is not a source code file and contains no application logic. It was generated during debugging to identify valid free model IDs. This file has no impact on the application runtime and does not require verification as a "fix". It is included here for completeness of the audit trail only.

3. Complete Issue Taxonomy

This chapter catalogs all issues discovered during the comprehensive codebase scan, organized by severity. Each issue includes the exact file path, line number where applicable, a description of the problem, and the potential impact on the application. The issues span security vulnerabilities, data integrity problems, mock data in production, architectural concerns, and UI/UX deficiencies.

3.1 Critical Issues (Fix Immediately)

#	File	Severity	Issue
1	swap-component.tsx (L12-25)	CRITICAL	Entire swap page uses hardcoded static prices for 12 tokens. PRICES object contains fake values (SOL: 145.23) that never update. Users see fabricated data as if real.
2	swap-component.tsx (L50-63)	CRITICAL	MOCK_BALANCES uses fake wallet balances (SOL: 42.5, USDT: 3200.0). No real wallet connection exists. Users could mistake this for real trading.
3	swap-component.tsx (L73-90)	CRITICAL	useMockWallet() hook returns a hardcoded address (7xKXtg2...). No real Solana wallet integration (Phantom, Solflare).
4	swap-component.tsx (L103-158)	CRITICAL	getSwapHistory() returns hardcoded fake swap history with fabricated dates and amounts. History is stored in localStorage only.
5	shared/data/index.ts (L318-323)	CRITICAL	getWatchlistData fetches ALL watchlist_items WITHOUT user_id filter. Every user sees every other user's watchlist items - data leak between users.

Table 2: Critical Issues

The swap component issue is the most severe: the entire page presents itself as a functional DEX swap interface with token selection, amount input, and transaction history, but every piece of data is fabricated. There is no Jupiter DEX integration, no real wallet connection via Solana wallet adapters, and no real transaction execution. A user could attempt to "swap" tokens, see a "successful" transaction in their history, and believe the trade was executed when nothing actually happened on-chain.

The watchlist data leak is equally critical from a security perspective. The getWatchlistData function queries the watchlist_items table without any user_id filter or JOIN to the watchlists table. The comment in the code acknowledges this: "watchlist_items doesn't have user_id - join via watchlists" but NO join is actually performed. This means User A can see User B's private watchlist, including any tokens they are monitoring for trading signals.

3.2 High Severity Issues

#	File	Severity	Issue
1	server/api/_security.ts (L95-99)	HIGH	requireAuth() only checks for "Bearer " prefix - never validates the actual JWT token. Any string starting with "Bearer " passes authentication. Complete auth bypass.
2	components/vixor/EquityChart.tsx (L85)	HIGH	Static SVG gradient ID "equityGradient" - if multiple EquityChart instances render on the same page, they share the same gradient ID causing color override.
3	shared/data/index.ts (L1317-1326)	HIGH	scanArbitrage() returns empty mock response. The entire arbitrage feature is a stub returning {opportunities: [], mode: "mock"}. Arbitrage page shows nothing functional.

Table 3: High Severity Issues

The authentication bypass in `_security.ts` is a serious security vulnerability. The `requireAuth` function at line 95-99 checks only that the Authorization header starts with "Bearer " and returns true without ever validating the token against Supabase JWT or any other verification mechanism. This means any unauthenticated request with the header Authorization: Bearer anything will pass authentication. All API routes that rely on this middleware are effectively unprotected.

3.3 Medium Severity Issues

#	File	Severity	Issue
1	discover-forex-data.ts (L91)	MEDIUM	Server function has NO <code>requireSupabaseAuth</code> middleware - forex discover data is publicly accessible to unauthenticated users.
2	routes/_authenticated/index.tsx (L1190)	MEDIUM	<code>getHomeMarketData</code> server function has no auth middleware - market data endpoint is unauthenticated.
3	analysis-id-component.tsx (L138-142)	MEDIUM	Analysis polls at 3-second intervals when status is "processing". If analysis gets stuck, this polls indefinitely with no timeout mechanism.
4	server/api/_security.ts (L54-77)	MEDIUM	In-memory rate limiting (Map) does not work across serverless instances (Vercel). Each function invocation gets its own Map, defeating rate limits.
5	routes/_authenticated/discover.tsx (L1455)	MEDIUM	Text "Mock data" rendered in forex tab UI even though forex data is now live. Stale comment misleads users.
6	routes/_authenticated/index.tsx (L1247)	MEDIUM	<code>btcDominance</code> is hardcoded to 0 - never actually fetched. Comment admits this limitation.
7	routes/_authenticated/swamp-component.tsx (L180)	MEDIUM	<code>formatAmount</code> uses <code>PRICES[symbol]</code> to determine decimals. For tokens not in the hardcoded <code>PRICES</code> map, comparison returns undefined causing potential NaN.

Table 4: Medium Severity Issues

3.4 Low and Informational Issues

#	File	Severity	Issue
1	index.tsx (L281, 427, 613)	LOW	Multiple "as any" type assertions to bypass router type-checking. Weakens TypeScript safety.
2	index.tsx (L1232-1239)	LOW	Binance API response typed as "any" - no runtime validation of API response shape.
3	signals.tsx (L276-277)	LOW	Sound plays for ALL new signals on first load. Refetch with re-sort could trigger false "new signal" sounds.
4	HunterScoreCard.tsx (L124-125)	LOW	<code>useEffect</code> runs mutation on mount with empty deps + <code>eslint-disable</code> . Stale data if props change.
5	TradingViewTickerView.tsx (L30-37)	LOW	Hardcoded default symbol list (6 symbols baked in). Cannot be customized without prop override.
6	binance-ws.ts (L272-274)	INFO	<code>symbolToPair</code> uses <code>lastIndexOf("USDT")</code> - for symbols like "BSUSDT" would split incorrectly.
7	server/api/_security.ts (L88-93)	INFO	<code>ADMIN_API_KEY</code> has no <code>.env.example</code> entry. May be unset, making admin routes inaccessible.

Table 5: Low and Informational Issues

4. Original 13 Issue Categories - Status Mapping

The user originally reported issues across 13 categories based on application screenshots. This section maps each original category to its current status, identifying which specific code files and lines are responsible, and what KIMI K3 needs to fix. None of the 13 categories have been fully resolved by the 3 backend fixes.

#	Category	Status	Root Cause / Key Files
1	Top Bar Inconsistencies	UNFIXED	Inconsistent layout/spacing across navbar. Check AppShell and layout components.
2	Static/Dead Widgets on Home	UNFIXED	btcDominance hardcoded to 0 (index.tsx L1247). Some widgets may still use static data.
3	Mock Data in Production	UNFIXED	swap-component.tsx is entirely mock. scanArbitrage returns mock. "Mock data" text visible in discover.
4	No Real Charts	UNFIXED	TradingViewMiniChart uses external iframe embed. No native charting library integration.
5	Unprofessional Analysis UI	UNFIXED	Analysis result display needs review. Check analysis-id-component.tsx for layout issues.
6	Wrong Analysis Results	PARTIAL	Model ID fixed (gemma-4-31b-it:free). But model quality may still produce inaccurate SMC analysis.
7	Mock Analysis Output	PARTIAL	Local fallback engine exists but generates synthetic data, not real analysis. Fallback = mock.
8	Wrong AI Copilot Approach	UNFIXED	MOXI should be an AI brain/agent, not a chat copilot. Architecture needs fundamental redesign.
9	Broken Discover Page	PARTIAL	Crypto discover works (DexScreener). Forex works (TwelveData). "Mock data" label still visible.
10	Forex Crash	FIXED	Forex data now fetched via TwelveData API. discover-forex-data.ts has Promise.allSettled error handling.
11	Broken Token Detail Pages	UNFIXED	token-symbol-component.tsx needs review. Holder/transaction data missing for Solana tokens.
12	Missing Holder/Transaction Data	UNFIXED	No on-chain data integration for Solana token holders or transaction history. Need Helius/RPC integration.
13	Empty Token Pages	UNFIXED	Tokens without DexScreener/Binance data show empty pages. Need fallback UI for unknown tokens.

Table 6: Original 13 Issue Categories Status

5. Security Vulnerabilities

Beyond the functional bugs and UI issues, the audit identified several security vulnerabilities that could compromise user data, allow unauthorized access, or mislead users about the state of their transactions. These vulnerabilities require immediate attention before any public deployment or user onboarding.

5.1 Authentication Bypass

The `requireAuth` function in `server/api/_security.ts` (line 95-99) implements a completely ineffective authentication check. It only verifies that the `Authorization` header contains the string `"Bearer "` as a prefix, without ever validating the actual token against Supabase's JWT verification endpoint or decoding the JWT payload. Any HTTP client can bypass authentication by including the header `Authorization: Bearer x` in their requests. This means all API routes protected by this middleware are effectively public, including any route that uses `requireAuth` as a guard.

The fix requires implementing proper JWT verification: extract the token after `"Bearer "`, decode it using the Supabase JWT secret (from `SUPABASE_JWT_SECRET` env var), verify the signature and expiration, and extract the `user_id` from the payload. This should be a reusable middleware function that attaches the verified user information to the request context for downstream handlers to use.

5.2 Cross-User Data Leak

The `getWatchlistData` function in `src/shared/data/index.ts` (lines 318-323) performs a query on the `watchlist_items` table without any `user_id` filter. The code comment explicitly acknowledges that `"watchlist_items doesn't have user_id - join via watchlists"` but no `JOIN` operation is actually performed. This means every authenticated user receives ALL watchlist items from ALL users in the system. In a multi-tenant Telegram bot scenario, this is a critical data leak that violates user privacy and could expose trading strategies.

5.3 Missing Auth Middleware on Endpoints

Two server functions lack authentication middleware entirely. The `getHomeMarketData` function in the home page route and the `getLiveForexDiscoverData` function in `discover-forex-data.ts` are both created with plain `createServerFn` without `requireSupabaseAuth` middleware. While market data may seem non-sensitive, unauthenticated access allows enumeration of all traded pairs, their prices, and trading volumes, which could be used for competitive intelligence or rate limit abuse.

5.4 Ineffective Rate Limiting

The rate limiting implementation in `_security.ts` (lines 54-77) uses an in-memory `Map` to track request counts per IP. This approach is fundamentally broken in serverless environments like Vercel, where each function invocation runs in an isolated container with its own memory space. The `Map` is created fresh on every cold start, meaning rate limits reset constantly and provide no actual protection against abuse. A proper solution requires an external rate limiting store (Redis, Upstash, or Supabase) that persists across function invocations.

6. Architecture and Design Issues

6.1 Swap Component - Complete Simulation

The swap component at `src/routes/_authenticated/swap-component.tsx` represents the most significant architecture issue in the application. The entire page - prices, balances, wallet connection, swap execution, and transaction history - is a pure simulation with no backend integration whatsoever. The component maintains 12 hardcoded token prices in a `PRICES` constant (line 12-25), fake wallet balances in `MOCK_BALANCES` (line 50-63), a hardcoded Solana address in `useMockWallet()` (line 73-90), and fabricated swap history in `getSwapHistory()` (line 103-158). The swap history is persisted to `localStorage` only, meaning it is lost when the user clears their browser data.

This is particularly dangerous because the UI is designed to look like a real DEX swap interface. Users of a Telegram trading bot would naturally assume that the swap feature executes real on-chain transactions. The absence of any "DEMO" or "SIMULATION" disclaimer in the UI makes this a deceptive user experience. KIMI K3 should either: (a) implement real Jupiter DEX integration with proper wallet adapters, or (b) clearly label the entire page as a simulation/demo with visible disclaimers on every action.

6.2 Mock Data Proliferation

Beyond the swap component, mock and stub data exists in several other parts of the codebase. The `scanArbitrage` function returns a hardcoded empty response with mode: "mock". The arbitrage engine has mock DEX clients in `src/domains/arbitrage/mock/`. A dummy trading adapter exists at `src/domains/trading/gateway/adapters/dummy-adapter.ts`. While some of these may be intentional development scaffolding, they should be clearly separated from production code and never exposed to users. The "Mock data" text string still visible in the discover page's forex tab (line 1455) suggests incomplete cleanup from a previous development phase.

6.3 Analysis Pipeline Fragility

While the three-tier fallback (OpenRouter API, local engine, synthetic result) is now in place, the analysis pipeline has architectural weaknesses. The local engine (`runLocalAnalysis`) generates analysis based on mathematical indicators without real market context, and the synthetic fallback (`generateFallbackResult`) produces entirely fabricated entry/stop-loss/take-profit levels. Neither of these should be presented to users as real trading signals. The UI should clearly indicate when results come from the fallback engine versus the AI model, and ideally disable trade execution for synthetic results.

7. Actionable Prompts for KIMI K3

This chapter provides precise, actionable prompts that KIMI K3 can directly use to fix each issue. Each prompt includes the exact file path, the specific line(s) to modify, the current broken code, and the expected fix. These prompts are designed to be copy-pasted into KIMI K3 with minimal context required, enabling efficient parallel fixing of independent issues.

7.1 CRITICAL: Fix Authentication Bypass

File: src/server/api/_security.ts

Lines: 95-99

Current Code:

```
function requireAuth(req: Request): boolean {
  const auth = req.headers.get("authorization");
  if (!auth?.startsWith("Bearer ")) return false;
  return true; // BUG: never validates the actual token
}
```

Required Fix: Import Supabase's JWT verification. Extract the token, verify the signature using SUPABASE_JWT_SECRET, check expiration, and return the decoded user_id. Return false for invalid/expired tokens. Ensure all API routes that use requireAuth actually receive and use the verified user identity.

7.2 CRITICAL: Fix Watchlist Data Leak

File: src/shared/data/index.ts

Lines: 318-323

Required Fix: Add a JOIN between watchlist_items and watchlists tables, filtering by the authenticated user's ID. The query should be: SELECT watchlist_items.* FROM watchlist_items JOIN watchlists ON watchlist_items.watchlist_id = watchlists.id WHERE watchlists.user_id = :userId ORDER BY added_at DESC. Import and use requireSupabaseAuth middleware to get the current user ID.

7.3 CRITICAL: Fix or Disable Swap Component

File: src/routes/_authenticated/swap-component.tsx

Required Fix (Option A - Real Integration): Replace all mock data with real Jupiter DEX API calls. Integrate @solana/wallet-adapter for real wallet connections. Use Jupiter's quote API for real-time pricing and swap API for transaction execution. Store swap history in Supabase, not localStorage.

Required Fix (Option B - Safe Demo): If real integration is not ready, add prominent "DEMO MODE" banners to the page header, disable the swap execution button, and add a visible disclaimer: "This is a simulation. No real transactions are executed." Remove fake wallet address and show "Connect Wallet" as disabled.

7.4 HIGH: Fix EquityChart Gradient ID Collision

File: src/components/vixor/EquityChart.tsx

Line: 85

Current Code: `<linearGradient id="equityGradient">`

Required Fix: Generate a unique gradient ID using `React.useId()` or a prop-based suffix. Example: `id={`equityGradient-${pair}-${timeframe}`}` or use `useId()` hook to create a unique identifier per component instance.

7.5 HIGH: Fix scanArbitrage Stub

File: src/shared/data/index.ts

Lines: 1317-1326

Required Fix: Either implement real arbitrage scanning using the existing arbitrage engine in `src/domains/arbitrage/`, or remove the arbitrage feature from the UI entirely. If keeping it, connect `scanArbitrage` to the real engine that fetches prices from multiple DEXs (Jupiter, Raydium) and calculates profit opportunities. If removing, show a "Coming Soon" placeholder in the UI.

7.6 MEDIUM: Remove "Mock Data" Label from Discover

File: src/routes/_authenticated/discover.tsx

Line: 1455

Required Fix: Remove the hardcoded "Mock data" text string from the forex tab UI. Since forex data is now live via TwelveData API (fetched through `discover-forex-data.ts` with `Promise.allSettled` error handling), this label is stale and misleading. Replace with proper loading/empty states consistent with the crypto tab.

7.7 MEDIUM: Add Auth Middleware to Server Functions

Files: `src/routes/_authenticated/index.tsx` (`getHomeMarketData`),

`src/routes/_authenticated/discover-forex-data.ts` (`getLiveForexDiscoverData`)

Required Fix: Add `.middleware([requireSupabaseAuth])` to both server functions. Import `requireSupabaseAuth` from the auth middleware module. This ensures only authenticated Telegram users can access market data endpoints.

7.8 MEDIUM: Add Analysis Polling Timeout

File: `src/routes/_authenticated/analysis-id-component.tsx`

Lines: 138-142

Required Fix: Add a maximum polling duration (e.g., 5 minutes). Track the start time when polling begins, and if the analysis is still in "processing" status after 5 minutes, stop polling and show a "Analysis timed out" error state with a retry button. This prevents infinite polling loops when the analysis backend gets stuck.

8. Architecture Recommendations for Clean Structure

8.1 Remove All Mock Data from Production Paths

The most impactful architectural improvement is to establish a clear boundary between development scaffolding and production code. All mock data, stub functions, and simulation components should be either moved to a dedicated `src/mocks/` directory with explicit import guards, or removed entirely. Any component that renders mock data should check an environment flag (e.g., `VIXOR_DEMO_MODE=true`) and display clear visual indicators when operating in demo mode. This prevents the current situation where production users cannot distinguish between real and fake data.

8.2 Centralize Authentication Middleware

Create a single, robust authentication middleware module that handles JWT verification, user extraction, and role-based access control. All server functions should use this centralized middleware instead of implementing ad-hoc auth checks. The middleware should be applied at the route level in TanStack Start's file-based routing system, ensuring that no endpoint can accidentally be left unprotected. Rate limiting should be implemented using an external store (Upstash Redis is free tier compatible) rather than in-memory Maps.

8.3 Implement Proper Error Boundaries and Fallbacks

The application needs React error boundaries at the route level to catch rendering errors gracefully instead of showing blank screens. Each major route should have an error boundary that displays a user-friendly error message with a retry button. API calls should have standardized error handling that distinguishes between network errors, authentication errors, rate limit errors, and data errors, showing appropriate UI for each case. The current approach of silently failing (e.g., empty catch blocks in `useLivePrices`) should be replaced with visible error indicators that help users understand what went wrong.

8.4 Data Layer Consolidation

The data access layer in `src/shared/data/index.ts` is a large monolithic file that mixes concerns from multiple domains (watchlist, trades, arbitrage, signals, analysis). This should be decomposed into domain-specific data access modules under each domain directory (e.g., `src/domains/watchlist/data.ts`, `src/domains/trades/data.ts`). Each module should export only the functions relevant to its domain, and each should include proper type validation (using Zod schemas) on both inputs and outputs to prevent data integrity issues.

9. Build and CI Notes for KIMI K3

When making changes to the VIXOR codebase, KIMI K3 must ensure all modifications pass the project's build and CI pipeline. The build command is `pnpm run build`, and the CI runs `pnpm eslint src/server/` followed by `pnpm tsc --noEmit`. Any introduced TypeScript errors or ESLint violations will block the build. Below are critical notes for safe code modification.

- `pnpm PATH`: Every bash command must include `export PATH="$HOME/.npm-global/bin:$PATH"` before running `pnpm` commands, as `pnpm` is installed in a custom location.

- Design Tokens: Dark background #08090C, primary #6366F1, bullish #22D3A6, bearish #FB4667, gold #F0C419. Do not introduce new colors without updating the design token system.
- AssetRegistry: Always add new trading pairs to src/shared/asset-registry/types.ts first. Never hardcode pair names or Binance symbols in component code.
- useLivePrices: Always use AssetRegistry.find() (not .get()) for user-facing lookups where unknown pairs should be handled gracefully.
- Server Functions: Always wrap with .middleware([requireSupabaseAuth]) for authenticated endpoints. Use createServerFn with proper method specification.
- API Responses: Validate all external API responses with Zod schemas before using the data. Never cast API responses to any type.
- Environment Variables: Available keys include SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY, TELEGRAM_BOT_TOKEN, TWELVEDATA_API_KEY, FINNHUB_API_KEY, OPENROUTER_API_KEY, CRON_SECRET. Never commit real keys to source control.